

区块链基础 - Lecture 5

5 FLP定理证明

本章讲义和PPT和上一章是同一份，参考第4章的讲义和PPT

5.1 FLP定理归约到两条引理 - Reducing The FLP Impossibility Theorem to Two Lemmas

我们继续看第二条引理，和第一条引理很类似，但更为巧妙。这条引理的作用类似归纳证明中的归纳步骤，它以一个模糊配置作为输入，并输出一个更长的模糊配置序列。合用这两条引理，就能够构造出我们所需要的无限模糊配置序列，引理1负责启动这个序列，然后不断引用引理2来生成后续的所有部分。

实际上，之前的证明忽略了一个微妙的问题。也正是这个微妙之处，使得第二个引理的陈述要比你预期的更复杂一点（如下图 Figure 3）。

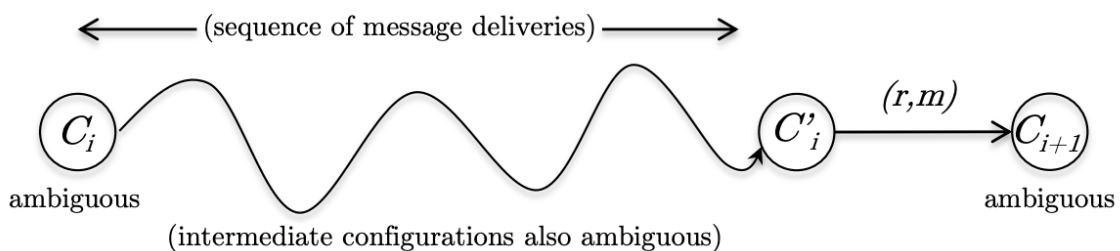


Figure 3: Lemma 8.1: extending a sequence of ambiguous configurations (from C_i to C_{i+1}) while delivering the message (r, m) last.

引理2（扩展一个模糊序列）：设一个固定的确定性协议 π ，使其在终止时满足一致性和有效性（ $f = 1, n \geq 2$ ）。令 C_i 为一个模糊配置， (r, m) 为 C_i 消息池中一条消息。则存在一个消息投递序列使得：

(i) 消息投递序列的最后一步是投递消息 (r, m) ；

(ii) 该序列执行完成后，系统到达的状态是一个模糊配置 C_{i+1} 。

或许你会疑惑，为什么不用一个更简单的引理代替：对于任意模糊配置 C_i ，总是存在一条消息交付后能够使系统到达的状态是另一个模糊配置 C_{i+1} 。

为什么我们要强行让引理适用于任意一条出现在 C_i 消息池中的消息 (r, m) ？

因为上述简单版本的引理是平凡成立的，从引理1所述初始模糊配置 C_0 出发，我们可以投递一个由无限多的空消息构造的消息序列（dummy message，即形如 $(r, \perp)$ 的消息）。空消息对系统不会产生任何影响，系统接收到空消息后不做任何决策，因此整个过程中产生的所有配置都是模糊的，系统依然保持模糊状态。如果我们反复引用简单版本的引理，就会得到一个敌手从未投递过任何有实际影响的消息（非平凡的）序列。显然，如果允许敌手这么做，那么达成共识是平凡不可能的。正因如此，在4.2节对异步模型的定理中，我们加上了最终可达性约束：每一条被添加到消息池中的消息，最终都必须被投递到它的目标接收者。

引理2表述中引入的额外的复杂性正是为了应对这个问题，它使我们能构造出一个满足消息最终可达性约束的无限配置序列。

结合引理1和引理2，我们可以对FLP给出下述证明：

设一个固定的确定性协议 π ，使其在终止时满足一致性和有效性（ $f = 1, n \geq 2$ ）。显然的，第一步我们引用引理1，选择一个初始模糊配置 C_0 。然后我们希望一次又一次地引用引理2，使得系统永远处于模糊状态，不可终止。每次引用引理2时，我们需要从当前的消息池中选择一条消息 (r, m) ，那我们要怎么选呢？

先把引理2放一边，假设我们暂时不关心是否保持模糊性，而只关心如何保证每条消息最终被投递。一个简单的解决方案是使用FIFO（First In First Out，先进先出）策略，即总是优先交付消息池中最早被发送的消息，即便这有可能会破坏系统的模糊性。这样一来，每当一条新消息被加入当前的消息池 M 时，我们就知道它将在至多 $|M|$ 次迭代后被投递（ $|M|$ 表示消息池中的消息总数量）。由于 M 是有限的（我们规定每轮迭代中每个节点只能发送有限条消息），这样能保证每条消息有明确的上界等待时间，即每条消息都能在有限步内被投递，避免消息饥饿问题（message starvation，指某一条消息永远在消息池中等待）。

引理2的表述方式使我们能够在两种极端方案之间找到一个这种策略：

- 一种是始终保持模糊状态，但不保证消息最终被投递的平凡解，比如只投递空消息；
- 另一种是保证消息最终都被交付但无法确保系统始终处于模糊状态的FIFO策略。

关键思想是，在保持模糊性的前提下，尽可能模拟FIFO的消息交付顺序。也就是说，我们想要模仿FIFO，尽可能的优先投递旧消息，但如果这条旧消息会破坏系统模糊性，那就先暂且搁置，投递其他的消息，等到时机合适的时候再送出。

具体来说，我们想要定义这样的模糊配置序列：

- 根据引理1，定义初始模糊配置 C_0 ；
- 对 $i = 0, 1, 2, \dots$:
 - 设 (r, m) 为 C_i 之消息池中的最旧之消息；
 - 根据引理2，定义 C_{i+1} 为相对于当前配置 C_i 和消息 (r, m) 的某个仍保持模糊的后继配置。

这个构造过程会构造出一系列子序列，在某些 C_i 和 C_{i+1} 之间可能会存在上亿个中间配置。但无论怎样，把这些子序列拼接起来，整体上仍然是一个配置序列。如图Figure 4。

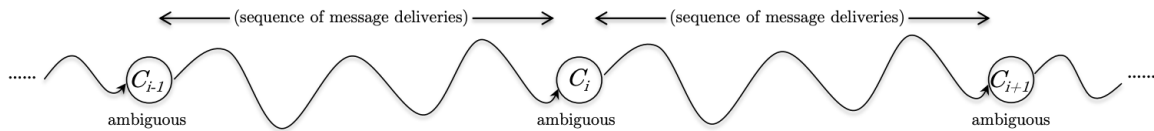


Figure 4: Repeated applications of Lemma 8.1 produces an arbitrarily long sequence (equivalently, sequence of sequences) of ambiguous configurations.

在 C_i 和 C_{i+1} 之间的这一段子序列，本质上是在拖延消息 (r_i, m_i) 的投递。这段子序列可以投递任意不会对系统模糊性造成破坏的消息，直到某一个时刻投递 (r_i, m_i) 也不会破坏模糊性再投递这条消息。

根据引理1和引理2，所有的 C_i 以及子序列中的中间配置，都是模糊配置。因为若非如此，系统的模糊性就会被破坏。这个序列同时也满足消息的最终可达性约束，与FIFO策略类似，如果某条消息 (r, m) 在配置 C_i 或在该配置之前被加入消息池 M ，那么它最迟会在配置 $C_{i+|M|}$ 处被投递。每次引用引理2，都会投递至少一条 $|M|$ 条消息中排在 (r, m) 之前的消息。因此，在至多 $|M|$ 次引用引理2后，消息 (r, m) 就会排到最前面，成为下一条被投递的消息。□

有时 (r, m) 可能会被更早地投递。例如，某次引用引理2的过程中，目标是投递一个更早的消息 (r', m') ，但需要先投递 (r, m) 才能确保投递 (r', m') 不会破坏模糊性。

现在我们已经知道引理1是成立的（引理1的证明在4.6节），并且联合引用引理1和引理2能够推出FLP定理。最后，我们来证明引理2（很难搞的证明）。

5.2 完成FLP定理证明 - Completing the Proof of the FLP Impossibility Theorem

为了证明引理2，不妨设一个固定的确定性协议 π ，使其在终止时满足一致性和有效性（ $f = 1, n \geq 2$ ），令 C_i 为 π 的一个配置，令 (r, m) 为 C_i 消息池中的一条消息。我们需要证明的是， (r, m) 既能够最终被投递，又可以保持系统的模糊性。这期间可能会投递上亿条其他消息。

5.2.1 证明准备 - Proof Setup

我们将配置分为三类，和定义4-2一样，这里也以固定的 π 协议为基础。但不同的是，这次还要相对于一个固定的配置 C_i 和消息 (r, m) 进行分类。

如果在配置 C_i 下直接投递 (r, m) 就能得到另一个模糊配置，那证明就完成了。因此我们假设一种相反的情况：立刻投递 (r, m) 会导致系统进入0-配置，即最终输出全0的结果（如是使系统进入1-配置，则证明是完全对称的，只需互换0和1即可）。

下面是我们需要定义三类配置：

定义5-1 (0*-、1*-、模糊*-配置) 设 C 为一个可由 C_i 出发，通过投递一系列非 (r, m) 消息而达到的配置。配置 C 可分为如下三类：

- 0*-配置：在配置 C 下投递 (r, m) 会使系统进入0-配置；
- 1*-配置：在配置 C 下投递 (r, m) 会使系统进入1-配置；
- 模糊*-配置：在配置 C 下投递 (r, m) 会使系统进入模糊配置。

由于任意一个配置必定是一个0-、1-或模糊配置，因此任意一个从 C_i 出发且尚未投递 (r, m) 的后继配置也必定是0*、1*或模糊*-配置。

一个0*-配置必定是一个0-配置（此时投递 (r, m) 不会影响结果）或一个模糊配置（一旦投递 (r, m) ，敌手所有能达成全1结果的策略会全部失效）。

一个模糊*-配置必定是一个模糊配置，它是一个特例，能够在模糊配置下投递 (r, m) 后依然保持模糊性。

带有星号*的配置是会受到 (r, m) 影响的配置，要注意和定义4-2的区别

利用上述定义，我们可以很清晰地重述我们的目标和假设：

- 引理2的结论：存在一个模糊*-配置。即，存在一种消息投递方式，使得在其之后再投递 (r, m) 会使系统进入模糊配置；
- 假设： C_i 是一个0*-配置。如果 C_i 是一个模糊*-配置，则证毕；如果它是一个1*-配置，那么我们只需在后续证明中将0和1互换即可。

5.2.2 搜索非0*-配置

通过广度优先搜索 (**breadth-first search, BFS**)：在4.5.2节中我们提到过，FLP定理的证明可以看作是在一个巨大的有向图中寻找一条无限的路径。图中的每一个顶点对应一个配置，每条边对应投递一条消息引起的状态转换。不妨试想将一个初始的模糊配置（且是0*-配置） C_i 作为顶点，由此出发进行BFS搜索，我们限定在搜索过程中忽略所有消息为 (r, m) 的边。根据定义5-1，搜索过程中遇到的每个配置，都是0*-配置、1*-配置和模糊*-配置中的一个。

逃离0*-配置：我们可以断言，在搜索过程中，必然会遇到一个非0*-配置。

证明：如果搜索只能遇到0*-配置，那么无论在什么时候投递 (r, m) ，其结果必定是进入0-配置，即最终输出为全0。在异步模型的最终可达性约束下，敌手最终必须投递 (r, m) ，则系统最终必定输出全0。亦即，配置 C_i 的输出结果为全0，这与 C_i 是一个模糊配置的假设矛盾，故上述断言成立。

一个候选的模糊*-配置：我们还需要排除一种可能，即这次BFS搜索始终只会遇到0*-配置和1*-配置。换一句话说，我们可以断言，在搜索过程中，必然会遇到模糊*-配置。

证明：根据上述证明，在这次BFS搜索过程中，我们必然会遇到一个非0*-配置，令这个配置为 Y ，且令 Y 为距离 C_i 最近的非0*-配置。即，从 C_i 出发，经过最少次数的消息投递能到达 Y 。如果有多个这样的配置，任意择其一即可。令 X 为 $C_i \rightsquigarrow Y$ 的搜索路径中 Y 的直接前驱配置。设 (r', m') 为该搜索路径上的最后一条被投递的消息，即触发 $X \mapsto Y$ 状态转换的消息。如图Figure 5。

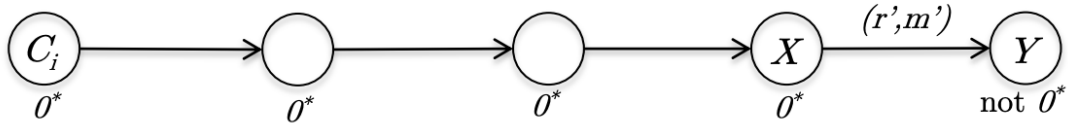


Figure 5: Let Y denote the non- 0^* -configuration that can be reached from C_i via the delivery of the fewest number of messages (none of which are (r, m)), and X its predecessor configuration on this shortest path.

由于本次搜索已经限定忽略消息为 (r, m) 的边，故消息 (r', m') 必须与 (r, m) 不同。也因如此，由于 (r, m) 在 C_i 的消息池中，则其也必然在 X 和 Y 的消息池中。配置 X 可能就是 C_i ，也可能不是。这取决于 C_i 的消息池中是否存在被立刻投递后能使得系统进入非 0^* -配置（ Y ）的消息。无论如何，由于 Y 是距离 C_i 最近的非 0^* -配置，且 X 是它的直接前驱，则 X 必然是一个 0^* -配置。

我们可以断言 Y 不可能是一个 1^* -配置（因此， Y 必然是一个模糊 * -配置），并且证明这个断言，来完成对引理2的证明。

同样的，我们可以使用反证法来证明该断言。假设 Y 是一个 1^* -配置，我们试着将注意力放到 X 和 Y 上，令 (r', m') 为触发 $X \mapsto Y$ 状态转换的消息。如图Figure 6。

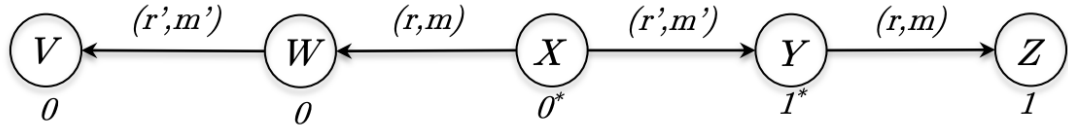


Figure 6: If Y is a 1^* -configuration, the order of the delivery of the messages (r, m) and (r', m') starting from configuration X dictates whether the protocol halts with the all-zero or the all-one outcome.

由于 X 是一个 0^* -配置，那么在 X 上投递 (r, m) 会进入一个 0 -配置，记为 W ；类似的，由于 Y 是一个 1^* -配置，则在 Y 上投递 (r, m) 会进入一个 1 -配置，记为 Z 。更进一步，我们考虑在 W 上投递仍未被投递的 (r', m') ，进入另一个配置，记为 V 。由于 W 是一个 0 -配置，则 V 也是 0 -配置。

一对关键的消息：Figure 6的核心要点在于这样的两个命题：

P1: 从配置 X （消息池中有 (r, m) 和 (r', m') 两条消息）开始，若先投递 (r, m) ，再投递 (r', m') ，则会得到一个 0 -配置（即配置 V ）；

P2: 从配置 X （消息池中有 (r, m) 和 (r', m') 两条消息）开始，若先投递 (r', m') ，再投递 (r, m) ，则会得到一个 1 -配置（即配置 Z ）。

也就是说，这两条消息的投递顺序决定了协议的最终结果是全0还是全1，聪明的你可能已经意识到正在悄然逼近的矛盾。我们将通过两种情况来证明，第一种是直接的、简单的，不涉及拜占庭节点；第二种类似我们在引理1中的证明。

Case 1: $r \neq r'$ ，即两条消息的接收者不同。如果这两条消息要发送给不同的接收者，那么没有任何节点知道它们被接收的相对顺序（需要注意的是，任意一个节点只知道自己的私有输入和自己所接收的消息序列，其行为完全由这些信息决定。节点无从知晓其他节点接收到哪些消息以及消息的接收顺序为何）。因此，调换这两条消息的接收顺序，并不会影响任何节点所接收的消息序列或其私有输入，所有节点的行为都将保持一致，也就是它们的输出都保持相同。这与上述命题P1和P2矛盾。

Case2: $r = r'$ ，即两条消息的接收者相同。如果两条消息的接收者是同一个节点 r ，那么有且只有节点 r 知道这两条消息投递的相对顺序。假设节点 r 恰好是一个拜占庭节点，它可以对其他节点隐瞒它收到 (r, m) 和 (r', m') 的真实顺序。即，拜占庭节点 r 可以采取两种策略：

- (i) 诚实地遵守 π 协议；
- (ii) 假装它先接收到了 (r, m) ，其余行为遵守 π 协议。

节点 r 以外的所有节点无法区分下述两个场景，因此这些节点会表现出完全一致的行为：

1. 节点 r 先收到了消息 (r, m) ，然后收到 (r', m') ，并且遵守 π 协议，即 r 采用策略(i)；
2. 节点 r 实际上先收到了消息 (r', m') ，然后才收到 (r, m) ，但它假装自己收到了 (r, m) ，即 r 采用策略(ii)。

由于除 r 以外的节点无法区分这两种场景，因此最后的输出结果是一致的，这与命题P1和P2矛盾。

综上所述，上述结论与配置 Y 是一个 1^* -配置矛盾，故配置 Y 是一个模糊*-配置。□

读者可以试证引理2中只有一个故障（崩溃）节点而非拜占庭节点的情况。

恭喜你！你已经成功完成了著名的FLP不可能性定理的证明，这个证明并不简单，但它能夯实你对分布式系统、区块链系统的底层认知。

首先，FLP定理并不是阻止你设计区块链协议，而是让你了解在设计时必须接受一种折中方案。例如，FLP告诉你，在网络遭受攻击时，所有区块链共识协议必须在consistency（所有节点保持同步）和活性（交易被持续处理）之间做出选择。这个选择是区块链协议设计决策中最为要紧的。BFT类协议（如Tendermint协议）在受到攻击时选择保持一致性，而最长链协议选择保持活性。不管你多么聪明，都无法设计出一个同时满足这两个性质的协议。

其次，FLP厘清了哪些假设是关键。例如，第2章和第3章说明了PKI假设很重要（至少在同步模型中），在这个可信设置下所能实现的结果，在缺少它的情况下是不可能达成的。类似的，FLP确认并形式化了这样一个直觉：在第2、3章中的同步模型假设非常关键，网络可靠性的程度从根本上影响了共识协议能达成的目标。

最后，FLP会引导你找到设计具有可证明保证协议所需的最小假设。

接下来我们会介绍部分同步模型（partially synchronous model），是一个介于不现实的同步模型和过于严苛的异步模型之间的一种解决方案。没有FLP，就没有人能想出这个折中模型，它可以说是评估区块链共识协议基本属性所需的最重要的模型。